

JDBC in Spring — Complete Notes

JDBC (Java Database Connectivity) is the standard Java API that lets Java code talk to a database — send SQL, get results back. Spring doesn't replace JDBC; it wraps it (mainly through `JdbcTemplate`) so you write less repetitive "boilerplate" code.

1. Big Picture — How JDBC Works



Think of it like ordering food at a restaurant:

- **DriverManager** = the phone directory that finds the right restaurant (database).
- **Connection** = the phone call that's connected.
- **Statement/PreparedStatement** = you telling the waiter what you want (the SQL order).
- **ResultSet** = the food that comes back to your table (the data).

2. Node: DriverManager

What it is: A built-in Java class (`java.sql.DriverManager`) whose only job is to create a `Connection` to a database using a JDBC driver.

Key points:

- Every database vendor (MySQL, PostgreSQL, Oracle...) provides its own **JDBC driver** — a small library that knows how to talk to that specific database.

- `DriverManager` manages a list of registered drivers and picks the right one based on the **connection URL** you give it.
- Since JDBC 4.0 (Java 6+), you usually **don't need to manually load the driver class** (`Class.forName(...)`) — it's auto-detected via the `META-INF/services` file inside the driver JAR.

Basic syntax (plain JDBC, no Spring):

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/mydb", // URL: protocol + host + port + db name
    "root",                             // username
    "password"                           // password
);
```

URL format breakdown:

```
jdbc:<subprotocol>://<host>:<port>/<database>
jdbc:mysql://localhost:3306/school_db
jdbc:postgresql://localhost:5432/school_db
```

In Spring: You almost never call `DriverManager` directly. Instead, Spring uses a `DataSource` (explained below) which internally manages connections more efficiently (via pooling). But it's important to know `DriverManager` because `DataSource` is built on the same underlying concept.

3. Node: Connection

What it is: An active session/link between your Java program and the database (`java.sql.Connection`). Everything (queries, updates, transactions) happens through this object.

Key responsibilities:

- Create `Statement`, `PreparedStatement`, `CallableStatement` objects.
- Manage **transactions** (commit/rollback).
- Hold database metadata (`getMetaData()`).

Important methods:

Method	Purpose
<code>createStatement()</code>	Get a plain <code>Statement</code>
<code>prepareStatement(sql)</code>	Get a <code>PreparedStatement</code>

Method	Purpose
<code>prepareCall(sql)</code>	Get a <code>CallableStatement</code> (for stored procedures)
<code>setAutoCommit(false)</code>	Turn off auto-commit to manage transactions manually
<code>commit()</code>	Save changes permanently
<code>rollback()</code>	Undo changes since last commit
<code>close()</code>	Release the connection back (very important — prevents leaks)

Transaction example:

```
conn.setAutoCommit(false);
try {
    // multiple SQL operations here
    conn.commit();
} catch (SQLException e) {
    conn.rollback();
}
```

Why closing matters: Connections are expensive (network handshake, memory). Not closing them causes **connection leaks**, which eventually crash your app. Always use **try-with-resources**:

```
try (Connection conn = dataSource.getConnection()) {
    // use conn
} // auto-closed here
```

In Spring: Spring's `JdbcTemplate` automatically opens and closes connections for you — this is one of the biggest pain points it removes.

4. Node: Statement

What it is: An object used to send **static/plain SQL** to the database (`java.sql.Statement`).

Key points:

- Created via `connection.createStatement()`.
- Good for simple, one-time queries where **no user input is involved**.
- **Not safe** for queries with dynamic values because you'd have to concatenate strings — leading to **SQL Injection** risk.

Methods:

Method	Use
<code>executeQuery(sql)</code>	For <code>SELECT</code> — returns a <code>ResultSet</code>
<code>executeUpdate(sql)</code>	For <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> , <code>DDL</code> — returns number of rows affected
<code>execute(sql)</code>	For SQL that might return either — returns boolean

Example:

```
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM students");
```

Why it's risky (SQL Injection example):

```
String name = userInput; // e.g. "'; DROP TABLE students; --"
stmt.executeQuery("SELECT * FROM students WHERE name = '" + name + "'");
// This could delete your whole table!
```

This is exactly why `PreparedStatement` exists.

5. Node: PreparedStatement

What it is: A **precompiled**, parameterized SQL statement (`java.sql.PreparedStatement`, which extends `Statement`). This is the **preferred** way to run SQL in real applications.

Key advantages over Statement:

1. **Prevents SQL Injection** — user values are sent separately from the SQL command, never concatenated into the query string.
2. **Better performance** — the SQL is compiled (parsed, optimized) once by the database and can be reused with different parameter values.
3. **Cleaner code** — no manual string concatenation.

Syntax — using `?` placeholders:

```
String sql = "SELECT * FROM students WHERE name = ? AND age > ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, "Riya"); // 1st ? → name
```

```
ps.setInt(2, 18); // 2nd ? → age
ResultSet rs = ps.executeQuery();
```

Common setter methods:

Method	For
<code>setString(index, value)</code>	VARCHAR/text
<code>setInt(index, value)</code>	integers
<code>setDouble(index, value)</code>	decimals
<code>setDate(index, value)</code>	dates
<code>setBoolean(index, value)</code>	true/false
<code>setNull(index, sqlType)</code>	NULL values

Insert/Update/Delete example:

```
String sql = "INSERT INTO students (name, age) VALUES (?, ?)";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, "Amit");
ps.setInt(2, 21);
int rowsInserted = ps.executeUpdate();
```

Batch operations (inserting many rows efficiently):

```
ps.setString(1, "A"); ps.setInt(2, 10); ps.addBatch();
ps.setString(1, "B"); ps.setInt(2, 20); ps.addBatch();
ps.executeBatch(); // sends all at once – much faster than one-by-one
```

Rule of thumb: Use `PreparedStatement` (or higher-level Spring tools) for anything involving user input. Reserve plain `Statement` only for fixed, hardcoded admin scripts.

6. Node: CallableStatement (bonus, often forgotten)

What it is: Used to call stored procedures inside the database (`java.sql.CallableStatement`, extends `PreparedStatement`).

```
CallableStatement cs = conn.prepareCall("{call getStudentById(?)}");
cs.setInt(1, 101);
ResultSet rs = cs.executeQuery();
```

Useful when business logic is written as a stored procedure inside the DB itself (common in enterprise/legacy systems).

7. Node: ResultSet

What it is: A table-like object holding the rows returned by a `SELECT` query (`java.sql.ResultSet`). You move through it row by row using a **cursor**.

Key points:

- The cursor starts **before the first row** — you must call `next()` before reading.
- `next()` returns `true` if there's a row, `false` when data ends.
- Columns can be read **by index** (1-based) or **by column name**.

Basic loop pattern:

```
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    int id = rs.getInt("id");        // by column name
    String name = rs.getString(2);  // by column index (2nd column)
    double marks = rs.getDouble("marks");
    System.out.println(id + " " + name + " " + marks);
}
```

Common getter methods: `getInt()`, `getString()`, `getDouble()`, `getDate()`, `getBoolean()`, `getObject()`, `getTimestamp()`.

ResultSet types (scrollability):

Type	Meaning
<code>TYPE_FORWARD_ONLY</code> (default)	Move forward only, one pass
<code>TYPE_SCROLL_INSENSITIVE</code>	Can move forward/backward; doesn't see live DB changes
<code>TYPE_SCROLL_SENSITIVE</code>	Can move forward/backward; reflects live DB changes

Concurrency modes:

Mode	Meaning
CONCUR_READ_ONLY (default)	Just read data
CONCUR_UPDATABLE	Can edit rows directly through the ResultSet using <code>updateRow()</code>

Metadata: `ResultSetMetaData` lets you inspect column names/types dynamically — useful for generic/reusable code:

```
ResultSetMetaData meta = rs.getMetaData();
int columnCount = meta.getColumnCount();
```

8. Node: Closing Resources & try-with-resources

Every JDBC object (`Connection`, `Statement`, `PreparedStatement`, `ResultSet`) holds a real resource (memory, DB cursor, network socket) that **must** be closed, or you get **resource leaks**.

Correct order (or use try-with-resources so order is automatic):

```
try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement ps = conn.prepareStatement(sql);
    ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        // process row
    }
} catch (SQLException e) {
    e.printStackTrace();
}
// all three auto-closed in reverse order, even if an exception occurs
```

This pattern alone eliminates most classic JDBC bugs.

9. Node: DataSource (the bridge into Spring)

What it is: An interface (`javax.sql.DataSource`) that is a more powerful, production-ready alternative to `DriverManager`.

Why prefer `DataSource` **over** `DriverManager`:

DriverManager	DataSource
Creates a brand-new connection every time (slow)	Uses a connection pool — reuses existing connections (fast)
No built-in pooling	Pooling libraries: HikariCP (Spring Boot default), Apache DBCP, C3P0
Manual configuration in code	Configurable via <code>application.properties</code> / <code>application.yml</code>
No JNDI support	Can be looked up via JNDI in app servers

Connection pooling concept: Instead of opening/closing a raw TCP connection for every request (expensive), a pool keeps a set of ready-to-use connections open and hands them out/takes them back — like a car rental service instead of buying a new car for every trip.

Spring Boot config example (`application.properties`):

```
spring.datasource.url=jdbc:mysql://localhost:3306/school_db
spring.datasource.username=root
spring.datasource.password=secret
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# HikariCP pool settings (optional tuning)
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.minimum-idle=2
```

Spring Boot auto-configures a `DataSource` bean from these properties — you don't write `DriverManager` code at all.

10. Node: JdbcTemplate (Spring's core JDBC helper)

What it is: `org.springframework.jdbc.core.JdbcTemplate` — a class that wraps raw JDBC to eliminate boilerplate: no manual `try/catch/finally`, no manual resource closing, no manual `SQLException` handling everywhere.

What it removes for you automatically:

- Opening/closing `Connection`
- Creating/closing `Statement` / `PreparedStatement`
- Iterating and closing `ResultSet`

- Converting checked `SQLException` → Spring's unchecked `DataAccessException`

Setup (Spring Boot auto-wires it once a `DataSource` bean exists):

```
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    // SELECT one row → map to object
    public Student findById(int id) {
        String sql = "SELECT * FROM students WHERE id = ?";
        return jdbcTemplate.queryForObject(sql, new StudentRowMapper(), id);
    }

    // SELECT many rows
    public List<Student> findAll() {
        String sql = "SELECT * FROM students";
        return jdbcTemplate.query(sql, new StudentRowMapper());
    }

    // INSERT / UPDATE / DELETE
    public int save(Student s) {
        String sql = "INSERT INTO students (name, age) VALUES (?, ?)";
        return jdbcTemplate.update(sql, s.getName(), s.getAge());
    }
}
```

RowMapper — converts a `ResultSet` row into a Java object:

```
public class StudentRowMapper implements RowMapper<Student> {
    @Override
    public Student mapRow(ResultSet rs, int rowNum) throws SQLException {
        Student s = new Student();
        s.setId(rs.getInt("id"));
        s.setName(rs.getString("name"));
        s.setAge(rs.getInt("age"));
        return s;
    }
}
```

Compare this to raw JDBC — no manual `Connection`, no manual `try/finally`, no manual `close()`.

`JdbcTemplate` handles all of that internally, using the exact same `DriverManager` → `Connection` → `PreparedStatement` → `ResultSet` flow underneath.

Other handy `JdbcTemplate` methods:

Method	Use
<code>queryForObject(sql, Class, args)</code>	Single value, e.g. a <code>COUNT(*)</code>
<code>queryForList(sql, args)</code>	List of <code>Map<String, Object></code> rows
<code>batchUpdate(sql, batchArgs)</code>	Efficient bulk inserts/updates
<code>execute(sql)</code>	DDL statements (CREATE TABLE, etc.)

11. Node: NamedParameterJdbcTemplate

What it is: A variant of `JdbcTemplate` that uses **named placeholders** (`:name`) instead of `?`, making SQL more readable when there are many parameters.

```
String sql = "SELECT * FROM students WHERE name = :name AND age > :age";
MapSqlParameterSource params = new MapSqlParameterSource();
params.addValue("name", "Riya");
params.addValue("age", 18);

List<Student> results = namedParameterJdbcTemplate.query(sql, params, new StudentRowMapper());
```

12. Node: Exception Handling — SQLException vs DataAccessException

Plain JDBC: Every JDBC method throws a **checked exception** `java.sql.SQLException`, forcing `try/catch` or `throws` everywhere — very noisy code.

Spring's improvement: `JdbcTemplate` catches `SQLException` internally and rethrows it wrapped as an **unchecked** `org.springframework.dao.DataAccessException` (and its many subclasses like `DuplicateKeyException`, `EmptyResultDataAccessException`, `DataIntegrityViolationException`).

Why this matters:

- You're not forced to catch exceptions you can't meaningfully recover from at every layer.
- Spring translates **vendor-specific error codes** (MySQL vs Oracle vs Postgres each throw different codes) into a **consistent set of exceptions**, so your business code doesn't need to

know which database is underneath.

13. Node: Transactions in Spring

Plain JDBC transaction (manual):

```
conn.setAutoCommit(false);
try {
    // multiple updates
    conn.commit();
} catch (Exception e) {
    conn.rollback();
}
```

Spring's improvement — declarative transactions with `@Transactional`:

```
@Service
public class StudentService {

    @Transactional
    public void transferMarks(int fromId, int toId, int marks) {
        studentRepository.deductMarks(fromId, marks);
        studentRepository.addMarks(toId, marks);
        // if any exception occurs, Spring auto-rolls back BOTH operations
    }
}
```

Spring wraps the method in a transaction proxy — commit happens automatically if the method completes normally, rollback happens automatically on a `RuntimeException`. You never touch `setAutoCommit`, `commit()`, or `rollback()` yourself.

14. Node: Isolation Levels & Concurrency (concept, applies to JDBC and Spring alike)

Isolation Level	Problem it prevents
READ_UNCOMMITTED	None (allows dirty reads)
READ_COMMITTED	Dirty reads

Isolation Level	Problem it prevents
REPEATABLE_READ	Dirty + non-repeatable reads
SERIALIZABLE	Dirty + non-repeatable + phantom reads (safest, slowest)

Set in Spring via `@Transactional(isolation = Isolation.READ_COMMITTED)`.

15. Node: Where Spring Data JPA fits in (context, not a replacement)

- **JDBC** = you write SQL yourself, get raw rows back.
- **JdbcTemplate** = same idea, less boilerplate.
- **Spring Data JPA / Hibernate** = an ORM layer built on top of JDBC — you work with Java objects/entities, and Hibernate generates the SQL and uses JDBC underneath to execute it.

So no matter how high-level your Spring app looks, **JDBC (Connection, Statement, ResultSet)** is **always running underneath** — Spring and JPA are layers of convenience on top of it, not replacements for it.

16. Quick Summary Table

Concept	Plain JDBC	Spring Equivalent
Get a connection	<code>DriverManager.getConnection()</code>	<code>DataSource</code> (pooled, auto-configured)
Run SQL	<code>Statement</code> / <code>PreparedStatement</code>	<code>JdbcTemplate.query()/update()</code>
Read results	<code>ResultSet</code> + manual loop	<code>RowMapper</code> (auto-mapped to objects)
Exceptions	Checked <code>SQLException</code> everywhere	Unchecked <code>DataAccessException</code>
Transactions	Manual <code>commit()/rollback()</code>	<code>@Transactional</code> annotation

Concept	Plain JDBC	Spring Equivalent
Resource cleanup	Manual <code>close()</code> / try-with-resources	Handled automatically

17. Mental Model to Remember

DriverManager → finds a driver, opens **ONE** connection (**old-school**)
 DataSource → pooled, managed connections (modern/Spring way)
 Connection → the **open** session/pipe **to** the DB
 Statement → send **static SQL** (risk: **SQL** injection)
 PreparedStatement → send parameterized, precompiled, safe **SQL**
 CallableStatement → **call** a stored **procedure**
 ResultSet → the **table of rows** coming back **from** a **SELECT**
 JdbcTemplate → Spring wrapper that automates **all of** the above
 @Transactional → Spring wrapper that automates **commit/rollback**